

Comprendre Kociemba

Un cours visuel pour coder le solveur en deux phases

Un cours pensé pour quelqu'un qui sait déjà simuler un Rubik's Cube en code, mais qui veut comprendre **vraiment** pourquoi Kociemba marche — et pas seulement recopier du code sans le comprendre.

Style : analogies visuelles, pas de code, un peu de vocabulaire mathématique toujours accompagné d'une explication intuitive.

Sommaire

Introduction — Pourquoi ce cours existe	p. 3
Chapitre 0 — Comprendre les orientations (co & eo)	p. 4
Chapitre 1 — Le cube comme un espace	p. 9
Chapitre 2 — L'idée géniale de Kociemba	p. 13
Chapitre 3 — Le sous-groupe G1, la salle d'attente	p. 17
Chapitre 4 — Les coordonnées, l'ADN du cube	p. 21
Chapitre 5 — IDA*, le GPS du solveur	p. 27
Chapitre 6 — Les tables de pruning, la carte routière	p. 32
Chapitre 7 — Assemblage complet de la machine	p. 36
Chapitre 8 — Roadmap pour coder	p. 39
Épilogue	p. 43

Introduction

Pourquoi ce cours existe

Tu as codé un simulateur de Rubik's Cube. Tu as codé un convertisseur qui transforme un cube visuel en état Kociemba. Maintenant tu veux coder le solveur lui-même, et tu te retrouves face à un mur : chaque tutoriel commence par te balancer du vocabulaire — groupe, permutation, coordonnée, pruning table, IDA* — sans jamais te faire **voir** ce que ça veut dire.

Ce cours prend le parti inverse. On ne va pas commencer par des définitions. On va commencer par des images mentales. Chaque concept abstrait de Kociemba a une contrepartie visuelle — un labyrinthe, une carte, un hall d'entrée, une bibliothèque. Quand tu auras ces images en tête, le code ne sera plus qu'une traduction.

***Idée clé :** Si tu ne peux pas dessiner un concept sur un tableau blanc à ton voisin, tu ne le comprends pas encore. Ce cours vise à te donner ce dessin pour chaque brique de Kociemba.*

Ce que tu sauras après ce cours

- Pourquoi Kociemba trouve des solutions courtes (~20 mouvements) là où le "couche par couche" en met 100.
- Ce qu'est un **groupe** et pourquoi le Rubik's Cube en est un.
- Pourquoi diviser le problème en deux phases est génial.
- Ce que sont les **coordonnées** (twist, flip, slice, permutations) et à quoi elles servent.
- Comment fonctionne **IDA*** et pourquoi c'est plus malin qu'une recherche en profondeur classique.
- Pourquoi on a besoin de **tables de pruning** précalculées.
- Dans quel ordre coder tout ça pour ne pas te perdre.

Ce que ce cours ne fait pas

Il n'y a aucun code JavaScript, C ou Python dans ce document. C'est volontaire. Tu écriras le code toi-même, et tu le feras mieux parce que tu auras compris avant.

Chapitre 0

Comprendre les orientations (co & eo)

Avant tout le reste, il faut absolument avoir en tête ce que sont les **orientations**. C'est le concept le plus contre-intuitif de la modélisation Kociemba, et si tu ne le comprends pas, tout le reste va rester flou. Prenons le temps.

0.1 — Position ≠ Orientation

Chaque pièce du cube (coin ou arête) a **deux informations distinctes** :

- **Où elle est** — sa position dans le cube. C'est ce que capturent *cp* (coin permutation) et *ep* (edge permutation).
- **Comment elle est tournée sur elle-même** — son orientation. C'est ce que capturent *co* (corner orientation) et *eo* (edge orientation).

Position et orientation sont indépendantes. Une pièce peut être à la bonne place mais mal orientée (tournée sur elle-même). Ça arrive tout le temps quand on résout un cube : tu réussis à mettre une pièce à sa place, mais elle regarde dans le mauvais sens.

***Idee clé :** L'orientation, ce n'est pas "vers où pointe la pièce". C'est un **compteur** qui dit combien de fois la pièce a tourné sur elle-même depuis sa position résolue.*

0.2 — Le compteur d'orientation

Un coin peut être tourné de 3 façons différentes (0, 1/3 de tour, 2/3 de tour). Son compteur **co** prend donc les valeurs **0, 1 ou 2**.

Une arête peut être tournée de 2 façons différentes (bien orientée ou à l'envers). Son compteur **eo** prend donc les valeurs **0 ou 1**.

Analogie visuelle — La boussole tatouée sur chaque pièce

Imagine que sur chaque coin du cube, tu as **tatoué une flèche invisible** qui pointe dans une direction précise. Sur un cube résolu, toutes ces flèches pointent "vers le haut ou vers le bas" (vers U ou D).

Quand tu fais bouger le cube, ces flèches suivent la pièce partout où elle va. Après un mouvement, la flèche peut :

- Toujours pointer vers U/D → **co = 0** (la pièce n'a pas tourné sur elle-même)
- Avoir tourné d'un cran dans un sens → **co = 1**
- Avoir tourné d'un cran dans l'autre sens → **co = 2**

La flèche, c'est le **sticker blanc ou jaune** du coin.

0.3 — Pourquoi le sticker blanc ou jaune ?

Parce que dans un cube résolu, tous les stickers W et Y sont sur les faces U et D. C'est notre "point de départ" naturel. Chaque coin contient exactement **un** sticker W ou Y (jamais zéro, jamais deux) — c'est notre marqueur unique par coin.

Après n'importe quel mélange, ce sticker se retrouve à l'une des trois positions du coin : sur une face U/D, sur une face F/B, ou sur une face L/R. Sa position **dans le coin** donne directement la valeur de **co**.

0.4 — Exemple concret : un mouvement F sur le coin UFR

Prenons le coin UFR (blanc-vert-rouge). Sur un cube résolu :

- Sticker blanc sur la face U (en haut)
- Sticker vert sur la face F (devant)
- Sticker rouge sur la face R (droite)

On fait un mouvement **F** (on tourne la face avant d'un quart de tour horaire). Le coin UFR se déplace vers la position DFR (en bas droite avant). Mais surtout, il **tourne aussi sur lui-même** pendant le trajet :

- Le sticker blanc, qui était en haut, est maintenant sur la face R (à droite)
- Le sticker vert reste sur la face F (il pivotait autour de l'axe F)
- Le sticker rouge est maintenant en bas

Le sticker blanc n'est plus sur U/D → le coin est **désorienté**. Il a tourné d'un cran. Selon le sens, **co = 1** ou **co = 2**.

Analogie visuelle — Le petit pois dans le beignet

Imagine un beignet troué (le coin) avec un petit pois collé dessus (le sticker W/Y). Tu fais rouler le beignet dans une gouttière (un mouvement du cube). Le beignet bouge de place, mais aussi le petit pois change de position sur le beignet : parfois il reste en haut, parfois il glisse sur le côté droit, parfois sur le côté gauche.

Le beignet, c'est la pièce. La position du beignet dans la gouttière, c'est **cp**. La position du petit pois sur le beignet, c'est **co**.

0.5 — Pourquoi les orientations sont vitales pour Kociemba

Voici l'observation qui rend Kociemba possible : **tous les mouvements ne changent pas les orientations de la même façon**. Certains mouvements les préservent complètement, d'autres non.

Les mouvements U, U', U2, D, D', D2 ne modifient jamais l'orientation d'un coin ni d'une arête. Une pièce peut changer de place, mais sa flèche invisible pointe toujours dans la même direction relative.

Les mouvements R, R', L, L' peuvent changer l'orientation des coins (co) mais pas des arêtes (eo).

Les mouvements F, F', B, B' peuvent changer aussi bien les coins que les arêtes.

Les demi-tours R2, L2, F2, B2 ne changent aucune orientation (elles se compensent).

Cette asymétrie est l'**idée directrice** derrière le choix des mouvements de la phase 2 chez Kociemba : ne garder que les mouvements qui préservent les orientations. On y reviendra en détail au chapitre 3.

0.6 — Le cas des arêtes : le flip

Pour les arêtes c'est plus simple : deux orientations possibles seulement. Une arête est soit **bien orientée** (eo = 0), soit **flippée** (eo = 1).

Une arête flippée, c'est une arête où les deux couleurs sont "à l'envers" par rapport à ce qu'on attendrait à sa position. Physiquement, la pièce a fait un demi-tour sur elle-même.

Analogie visuelle — La pièce de monnaie

Imagine chaque arête comme une pièce de monnaie posée à un endroit précis, avec un côté "face" et un côté "pile". Bien orientée, c'est "face" visible (eo = 0). Flippée, c'est "pile" visible (eo = 1). C'est binaire, il n'y a pas d'intermédiaire.

0.7 — Un invariant magique

Une propriété fondamentale du cube — imposée par sa mécanique physique — dit que :

- La **somme des co** (des 8 coins) est toujours divisible par 3.
- La **somme des eo** (des 12 arêtes) est toujours paire.

Autrement dit : tu ne peux jamais avoir un cube physique avec un seul coin mal orienté, ou une seule arête flippée. Ce serait impossible à obtenir avec des mouvements légaux. C'est un **invariant** du cube : quelque chose qui ne change jamais.

Conséquence pratique : quand on encode les orientations en coordonnées, on n'a pas besoin de stocker toutes les valeurs. Les 7 premières orientations de coins déterminent la 8^{ème} (contrainte de somme divisible par 3). Les 11 premières orientations d'arêtes déterminent la 12^{ème} (contrainte de somme paire). C'est pour ça que la coordonnée **twist** ira de 0 à 3^7-1 (et pas 3^8-1), et **flip** de 0 à $2^{11}-1$.

***Idée clé :** Résumé : co (0/1/2) et eo (0/1) sont des **compteurs de rotation propre**. Ils comptent combien la pièce a tourné sur elle-même, indépendamment de sa position. Kociemba tire toute sa puissance du fait que certains mouvements laissent ces compteurs inchangés.*

Chapitre 1

Le cube comme un espace

Avant de parler de Kociemba, il faut changer ta façon de voir le Rubik's Cube. Pas comme un jouet en plastique — comme un **espace** qu'on explore.

1.1 — Combien d'états possibles ?

Un Rubik's Cube 3×3 a exactement **43 252 003 274 489 856 000** configurations possibles. Quarante-trois milliards de milliards. Tu peux considérer que chaque configuration est un **point** dans un espace immense. L'état résolu est un point particulier au centre de cet espace.

Analogie visuelle — L'univers-cube

Imagine que chaque configuration possible du cube est une étoile dans un univers gigantesque. Il y a 43 milliards de milliards d'étoiles. L'étoile "cube résolu" est quelque part au milieu. Ton cube mélangé, c'est une autre étoile, très très loin de la première. Résoudre le cube, c'est trouver un chemin d'étoile en étoile qui mène du mélange au résolu.

1.2 — Les mouvements comme des flèches

Comment on passe d'une étoile à une autre ? Avec un **mouvement**. Il y a 18 mouvements possibles : U, U', U2, D, D', D2, R, R', R2, L, L', L2, F, F', F2, B, B', B2. Chaque mouvement est comme une **flèche** qui part d'une étoile et pointe vers une autre.

Depuis n'importe quelle étoile, tu as 18 flèches qui partent (une par mouvement). Et symétriquement, 18 flèches arrivent. L'espace du cube est un immense réseau connecté par ces flèches.

Analogie visuelle — Le métro à 43 milliards de stations

Imagine un système de métro fou où chaque station a exactement 18 lignes qui en partent. Tu es dans une station (ton cube mélangé). Tu veux aller à la station centrale (cube résolu). Chaque "trajet" que tu prends est un mouvement. Résoudre le cube en 20 mouvements, c'est trouver un chemin qui passe par seulement 20 stations.

1.3 — Le mot "groupe" en 2 minutes

En maths, on appelle cet ensemble "les configurations du cube + les mouvements qui les relient" un **groupe**. C'est un mot savant pour désigner quelque chose de très simple : un ensemble d'objets et une opération qui les combine.

Trois propriétés font qu'un ensemble est un groupe :

1. Une opération pour combiner deux éléments. Ici : appliquer deux mouvements l'un après l'autre. Si je fais R puis U, j'obtiens un nouveau "mouvement composé" RU.

2. Un élément neutre. Un mouvement qui ne change rien. Ici : "ne rien faire". Si tu fais R puis "rien", tu as juste fait R.

3. Chaque élément a un inverse. Pour chaque mouvement, il en existe un qui l'annule. R et R' s'annulent : R suivi de R' te ramène à l'état de départ.

Idée clé : Quand un mathématicien dit "le groupe du cube", il veut juste dire "l'ensemble des positions du cube, avec les mouvements comme façon de passer de l'une à l'autre". Rien de plus mystique.

1.4 — Pourquoi ça sert de penser en "espace"

Parce que résoudre le cube devient un problème de **chemin le plus court dans un graphe**. Et en informatique, on sait bien traiter ce genre de problème (BFS, A*, IDA*...). Le défi de Kociemba n'est pas de trouver un chemin — c'est de le faire vite dans un graphe de 43 milliards de milliards de nœuds.

Analogie visuelle — Pourquoi un GPS est possible

Un GPS trouve un chemin en France en quelques secondes alors qu'il y a des millions de routes possibles. Il ne teste pas toutes les routes une par une — il utilise une heuristique intelligente. Kociemba fait pareil sur le cube. On va voir comment.

Chapitre 2

L'idée géniale de Kociemba

Herbert Kociemba a eu en 1992 une idée qui a révolutionné la résolution du Rubik's Cube. Pas une idée technique — une idée **conceptuelle**. Elle tient en une phrase.

"Plutôt que d'aller directement du cube mélangé au cube résolu, passons par une étape intermédiaire."

2.1 — Le problème avec la recherche directe

Si tu voulais résoudre le cube "bêtement" avec un algorithme de plus court chemin classique, tu devrais explorer les 18 mouvements possibles depuis ton état, puis les 18 mouvements depuis chacun des états obtenus, etc. À chaque niveau, tu multiplies par ~15 (moins que 18 parce qu'on évite les mouvements redondants).

En 20 niveaux de profondeur (la longueur typique d'une solution optimale), ça fait $15^{20} \approx 33\,000\,000\,000\,000\,000\,000\,000$ chemins à explorer. Même les ordinateurs les plus puissants n'y arriveraient pas en un temps raisonnable.

2.2 — L'astuce du hall d'entrée

Kociemba a remarqué qu'il existe un **sous-ensemble** de configurations du cube qui a une propriété magique : depuis ce sous-ensemble, on peut résoudre le cube en n'utilisant qu'un nombre **limité** de mouvements — pas les 18, seulement 10.

Analogie visuelle — Le hall d'entrée d'un immeuble

Imagine que ton cube mélangé est une chambre au 42nd étage d'un immeuble immense (43 milliards d'étages). Tu veux atteindre une chambre spécifique (le cube résolu) qui est au 27nd étage. Au lieu de chercher un chemin direct de chambre à chambre (impossible), Kociemba dit :

« Descendons d'abord au hall d'entrée. Depuis n'importe quelle chambre, c'est faisable. Une fois dans le hall, on connaît tous les chemins pour atteindre la chambre-cible. »

Ce "hall d'entrée" — dans le vocabulaire Kociemba, on l'appelle **G1**. C'est un sous-groupe particulier de configurations du cube.

2.3 — Les deux phases

L'algorithme se décompose donc en **deux phases** :

Phase 1. Depuis le cube mélangé, trouver une séquence de mouvements qui amène le cube dans G1 (le hall d'entrée). On peut utiliser tous les 18 mouvements. Cette phase

prend en général entre 7 et 12 mouvements.

Phase 2. Depuis G1, résoudre complètement le cube. Cette fois-ci, seuls 10 mouvements sont autorisés (on verra pourquoi). Cette phase prend entre 10 et 18 mouvements.

Au total, on obtient des solutions de 20 à 30 mouvements. En comparaison, l'optimum absolu (dit "Nombre de Dieu") est **20 mouvements** — prouvé mathématiquement en 2010. Kociemba n'est pas optimal, mais il est extrêmement proche et surtout **rapide**.

2.4 — Pourquoi c'est plus rapide

Chaque phase individuellement a un espace de recherche **beaucoup plus petit** que l'espace complet. Au lieu d'explorer 15^{20} chemins, on explore 15^{10} chemins pour la phase 1, puis 10^{18} pour la phase 2. Les nombres restent grands, mais avec les bonnes optimisations (coordonnées + tables de pruning, on y viendra), chaque phase se résout en quelques millisecondes.

***Idée clé :** L'idée de Kociemba n'est pas mathématique — elle est **stratégique**.
Diviser pour régner : au lieu d'un problème énorme, deux problèmes plus petits qui, chacun, sont abordables.*

2.5 — Le prix à payer

Cette stratégie a un coût : les solutions ne sont pas **optimales**. Un cube pourrait peut-être être résolu en 18 mouvements en cherchant directement, mais Kociemba va peut-être trouver une solution en 23 mouvements (10 pour la phase 1 + 13 pour la phase 2). En pratique, c'est un excellent compromis.

Une amélioration classique consiste à lancer Kociemba plusieurs fois avec différentes stratégies de phase 1 et à garder la meilleure solution globale. C'est ce que font les solveurs modernes pour se rapprocher de l'optimum.

Chapitre 3

Le sous-groupe G1, la salle d'attente

Maintenant on va zoomer sur ce fameux "hall d'entrée" : G1. Qu'est-ce qui le rend spécial ? Pourquoi ce sous-ensemble et pas un autre ?

3.1 — La définition visuelle de G1

Un cube est dans G1 si et seulement si **trois propriétés** sont vraies :

1. **Toutes les arêtes sont bien orientées.** Toutes les valeurs de eo sont à 0.
2. **Tous les coins sont bien orientés.** Toutes les valeurs de co sont à 0.
3. **Les 4 arêtes de la tranche E** (les arêtes FL, FR, BR, BL, celles du milieu) sont **toutes dans la tranche E** — pas forcément à leur place exacte, juste dans la bonne tranche.

Notons ce qu'un cube dans G1 n'a pas besoin d'être : il n'est pas résolu. Les couleurs peuvent être totalement mélangées. Simplement, elles sont mélangées d'une **certaine façon**.

Analogie visuelle — Le hall d'entrée comme salle bien rangée

Imagine une bibliothèque. Le "cube résolu", c'est quand chaque livre est à sa place précise sur la bonne étagère. Le "cube mélangé", c'est quand tous les livres sont par terre en pagaille.

G1, c'est un **état intermédiaire** : chaque livre est **debout sur une étagère** (pas de livre par terre, pas de livre à l'envers) — mais pas forcément la bonne étagère ni le bon ordre. Depuis cet état, ranger devient beaucoup plus facile.

3.2 — Pourquoi ces trois propriétés ?

Kociemba a choisi ces propriétés parce qu'elles ont une caractéristique magique : elles sont **préservées** par un ensemble de mouvements bien précis. Lesquels ?

Les mouvements **U, U', U2, D, D', D2** (les tours des faces haut et bas) et **L2, R2, F2, B2** (les demi-tours des 4 faces latérales). Ça fait **10 mouvements**. C'est ce qu'on appelle les **générateurs de G1**.

Autrement dit : si tu es dans G1 et que tu appliques n'importe lequel de ces 10 mouvements, tu restes dans G1. Aucune arête ne se "dés-orient", aucun coin ne se "dés-orient", les arêtes du milieu restent au milieu. C'est ce qu'on appelle un **sous-groupe fermé** : un espace dont on ne peut pas sortir avec ces mouvements-là.

Analogie visuelle — Le jardin clôturé

Imagine un jardin délimité par une clôture. Tu es dedans (dans G1). Tu peux

marcher, courir, sauter dans tous les sens — tant que tu utilises les "chemins autorisés" (les 10 mouvements), tu ne franchis jamais la clôture. Tu restes dans le jardin.

En revanche, si tu utilises un "chemin interdit" (comme R, F', L...), tu sautes par-dessus la clôture et tu te retrouves dehors, dans le grand espace du cube.

3.3 — Ce que ça change concrètement

Regardons combien de configurations il y a dans G1. La réponse : **19 508 428 800**. Environ 19 milliards. C'est beaucoup, mais c'est **2 milliards de fois moins** que l'espace complet du cube (43 milliards de milliards).

Autrement dit : la phase 2 explore un espace 2 milliards de fois plus petit que la phase 1. C'est ce qui la rend faisable.

3.4 — La phase 1 vue depuis G1

Reformulons la phase 1 avec cette nouvelle compréhension : la phase 1 doit corriger les **trois propriétés qui définissent G1** :

- Orienter toutes les arêtes (mettre tous les eo à 0).
- Orienter tous les coins (mettre tous les co à 0).
- Amener les 4 arêtes de la tranche E dans la tranche E.

Elle n'a pas besoin de mettre les pièces à leur place finale, ni d'ordonner quoi que ce soit. Juste ces trois choses.

***Idée clé** : G1 est un état intermédiaire soigneusement choisi. Pas trop restreint (sinon la phase 1 serait aussi dure que le problème complet), pas trop permissif (sinon la phase 2 aurait trop de possibilités). L'équilibre parfait entre les deux.*

Chapitre 4

Les coordonnées, l'ADN du cube

Jusqu'ici, on a représenté un état du cube avec 4 tableaux : cp, co, ep, eo. C'est très lisible, mais c'est aussi **encombrant** pour un algorithme de recherche qui doit explorer des millions d'états. On va compresser ça.

4.1 — Le problème du poids

Imagine que ton solveur explore 10 millions d'états pour trouver une solution. Si chaque état occupe 40 octets (les 4 tableaux avec leurs valeurs), ça fait **400 Mo** de mémoire juste pour stocker les états visités. Et comparer deux états "tableau par tableau" est lent.

Il faut un moyen de représenter chaque état par **quelques entiers**. Un entier tient sur 4 octets, il se compare en une instruction machine, il s'utilise comme index dans une table. C'est l'idée des **coordonnées**.

Analogie visuelle — L'ADN du cube

Un être humain est complexe — des milliards de cellules, chacune avec ses fonctions. Pourtant, on peut résumer une personne par son ADN : quelques milliards de bases, mais qu'on peut aussi résumer par des "marqueurs" clés (groupe sanguin, couleur des yeux, etc.).

Les coordonnées, c'est pareil : quelques entiers qui capturent ce qui compte de l'état du cube pour la recherche.

4.2 — La recette générale

Pour chaque "aspect" du cube qui nous intéresse, on définit une **coordonnée** : une fonction qui prend l'état complet et renvoie un entier unique. Deux états différents doivent avoir des coordonnées différentes (bijection), et les coordonnées doivent être **compactes** (petits entiers).

Pour Kociemba, on définit **trois coordonnées pour la phase 1** et **trois autres pour la phase 2**. On les découvre ci-dessous.

4.3 — Les coordonnées de la phase 1

La phase 1 doit corriger trois choses : orientation des coins, orientation des arêtes, position des arêtes de la tranche E. Une coordonnée par chose.

Twist — orientation des coins

Chaque coin a une orientation (0, 1 ou 2). Il y a 8 coins. On pourrait naïvement dire : c'est un nombre à 8 chiffres en base 3. Mais il y a une contrainte : la somme des orientations est toujours divisible par 3 (invariant du cube). Donc **la 8^{ème} orientation est déterminée** par les 7 premières.

On encode donc les 7 premières comme un nombre en base 3 : **de 0 à $3^7 - 1 = 2186$** . Une coordonnée entière entre 0 et 2186 qui capture toute l'information d'orientation des coins.

Analogie visuelle — Le compteur kilométrique

Un compteur kilométrique à 7 roues, chacune de 0 à 2. Chaque configuration de roues donne un nombre unique entre 0 et 2186. Le **twist**, c'est ce compteur.

Flip — orientation des arêtes

Même principe. 12 arêtes, chacune orientée 0 ou 1. Contrainte : la somme est paire. Donc la **12^{ème}** est déterminée par les 11 premières. On encode les 11 premières en base 2 : de 0 à $2^{11} - 1 = 2047$.

Slice — position des arêtes de la tranche E

Il y a 12 emplacements pour les arêtes, et on veut savoir dans quels 4 emplacements se trouvent les 4 arêtes du milieu (FL, FR, BR, BL). C'est un problème combinatoire classique : combien de façons de choisir 4 éléments parmi 12 ? La réponse est **C(12,4) = 495**.

On numérote ces 495 combinaisons de 0 à 494. La **slice**, c'est ce numéro.

Analogie visuelle — Les jetons dans les cases

Imagine 12 cases alignées et 4 jetons rouges. Peu importe l'ordre des jetons entre eux, seule compte la question "dans quelles 4 cases sont-ils ?". Il y a 495 réponses possibles. La slice, c'est le numéro de la configuration actuelle.

4.4 — L'état de la phase 1 en 3 entiers

Un état du cube pour la phase 1 est donc entièrement décrit par le triplet (**twist, flip, slice**). Trois entiers qui pèsent 12 octets au total.

L'état-but de la phase 1 est **(0, 0, 0)** : tous les twists à 0 (coins orientés), tous les flips à 0 (arêtes orientées), slice à 0 (arêtes du milieu dans le milieu, dans un ordre canonique).

Attention : slice = 0 signifie que les 4 arêtes du milieu **occupent** les 4 slots du milieu, pas qu'elles sont à leur place précise. C'est bien la définition de G1.

4.5 — Les coordonnées de la phase 2

La phase 2 démarre déjà dans G1 (coins/arêtes orientés, tranche E au milieu). Ce qui reste à faire : permuter correctement les pièces. Trois nouvelles coordonnées :

Corner permutation : 8 coins à ordonner. $8! = 40\,320$ permutations possibles.

Edge permutation U/D : les 8 arêtes qui restent dans les couches U et D. $8! = 40\,320$.

Slice permutation : l'ordre des 4 arêtes du milieu entre elles. $4! = 24$.

Encoder une permutation en entier : chaque permutation peut être numérotée de 0 à $N!-1$ avec un procédé standard (Lehmer code, factorielle). Encore une fois, chaque état de phase 2 tient en 3 entiers.

***Idée clé** : Les coordonnées sont un **langage compressé** pour parler du cube. Elles font perdre la lisibilité (on ne "voit" plus le cube en regardant les nombres) mais gagnent en vitesse. C'est le prix à payer pour un solveur rapide.*

4.6 — Les tables de mouvements

Une fois qu'on a des coordonnées, on peut précalculer une **table** qui dit : "si tu es à la coordonnée X et que tu appliques le mouvement M, tu passes à la coordonnée Y". C'est un tableau bidimensionnel [coordonnée × mouvement → nouvelle coordonnée].

Ces tables se calculent une seule fois au démarrage du solveur (ou même à la compilation). Après ça, appliquer un mouvement devient **une simple lecture** dans un tableau : instantané.

Analogie visuelle — Le dictionnaire de traduction

Au lieu de recalculer à chaque fois "comment se dit 'chat' en anglais" en analysant la phonétique, on a un dictionnaire précalculé : mot français → mot anglais. Lecture instantanée. Les tables de mouvements sont ces dictionnaires pour Kociemba.

Chapitre 5

IDA*, le GPS du solveur

On a maintenant tout ce qu'il faut pour **décrire** l'état du cube et **appliquer** des mouvements rapidement. Il manque l'essentiel : comment on cherche le bon chemin ?

5.1 — Pourquoi pas BFS ou DFS ?

Si tu connais un peu les algorithmes de graphe, tu penses peut-être : "BFS (parcours en largeur), ça trouve le plus court chemin". Vrai en théorie. En pratique, sur le cube, BFS explore un nombre gigantesque de nœuds. Il faudrait garder en mémoire tous les nœuds à explorer — impossible.

DFS (parcours en profondeur) utilise peu de mémoire mais peut se perdre dans des branches infinies. Il faut le limiter en profondeur — c'est ce qu'on appelle DFS itératif approfondi (IDDFS) : on lance DFS avec profondeur max = 1, puis 2, puis 3... jusqu'à trouver.

IDA* = IDDFS + **heuristique**. C'est-à-dire : à chaque nœud, on utilise une estimation intelligente de la distance restante pour ne pas explorer des branches qu'on sait trop longues.

5.2 — La logique du GPS

Pense à un GPS qui cherche un chemin. À chaque intersection, il regarde toutes les routes possibles et évalue : "combien je viens de faire" + "combien il reste à faire (estimation)". Cette somme, c'est le **coût total estimé** du chemin.

Le GPS ne prend jamais une route dont le coût total estimé est plus grand que le meilleur chemin déjà trouvé. C'est la même logique dans IDA*.

Analogie visuelle — L'oiseau qui aide à couper court

Imagine que tu explores une forêt à pied et que tu cherches une clairière. À chaque arbre, un oiseau te dit : "la clairière est à au moins 500 mètres d'ici". Si tu es à 300 mètres du départ et qu'on te dit qu'il reste au moins 500 mètres, tu sais que ce chemin coûtera au moins 800 mètres. Si tu as déjà trouvé un chemin de 700 mètres, tu fais demi-tour.

L'oiseau, c'est l'**heuristique**. Elle ne doit jamais surestimer (sinon on couperait des bons chemins). Plus elle est précise, plus l'algorithme est efficace.

5.3 — L'algorithme IDA*

On fixe une profondeur maximale **threshold** (au début : l'heuristique appliquée à l'état initial). On explore l'arbre des mouvements en profondeur. À chaque nœud, on calcule :

$$f(\text{nœud}) = g(\text{nœud}) + h(\text{nœud})$$

Où :

- **$g(\text{nœud})$** = nombre de mouvements déjà faits pour arriver ici
- **$h(\text{nœud})$** = estimation du nombre de mouvements restants (l'heuristique)

Si **$f(\text{nœud}) > \text{threshold}$** , on abandonne cette branche. Sinon on continue à explorer. Si on ne trouve rien à ce threshold, on l'augmente d'un cran et on recommence.

5.4 — Pourquoi IDA* est adapté au cube

IDA* utilise **très peu de mémoire** (il ne garde en mémoire que le chemin actuel), et grâce à l'heuristique, il élague massivement l'arbre de recherche. Sur le cube, avec une bonne heuristique, il trouve des solutions en quelques millisecondes.

Reste une question critique : **comment obtenir cette heuristique ?** C'est là qu'interviennent les **tables de pruning**, le sujet du prochain chapitre.

***Idée clé :** IDA* est un algorithme simple à décrire mais puissant : un DFS itératif qui utilise une estimation intelligente pour couper les branches trop longues. Sur le cube, il permet de passer d'un problème "impossible" à un problème "instantané".*

Chapitre 6

Les tables de pruning, la carte routière

L'efficacité d'IDA* dépend entièrement de la qualité de son heuristique **h**. Une heuristique qui renvoie toujours 0 rend IDA* équivalent à IDDFS (lent). Une heuristique très précise le rend fulgurant. Voyons comment Kociemba fabrique une heuristique quasi-parfaite.

6.1 — L'idée : précalculer toutes les distances

Si on pouvait connaître, pour chaque état du cube, sa **distance exacte** à l'état résolu, on aurait une heuristique parfaite. Le problème : il y a 43 milliards de milliards d'états. Impossible à stocker.

En revanche, on peut précalculer la distance pour chaque valeur des **coordonnées** individuellement. Souviens-toi : les coordonnées ont chacune une petite plage de valeurs (2187, 2048, 495...). Pour chacune, on peut construire une table qui dit "depuis cette valeur de coordonnée, il faut au moins X mouvements pour la ramener à 0".

Analogie visuelle — La carte routière régionale

Un GPS qui devrait connaître exactement le trajet entre chaque paire de villes du monde aurait besoin d'une base de données colossale. Au lieu de ça, il stocke des **cartes régionales** : pour chaque région, la distance approximative à toutes les villes. En combinant plusieurs cartes régionales, on obtient une estimation globale.

Les tables de pruning, c'est exactement ça : plusieurs "cartes régionales" du cube, chacune peu coûteuse, qui combinées donnent une bonne estimation globale.

6.2 — Construction d'une table de pruning

On veut, par exemple, une table qui donne pour chaque valeur du **twist** (0 à 2186) la distance minimale au twist 0. Comment on la construit ?

On fait un **BFS en arrière**, depuis l'état-but. On part de twist = 0, distance = 0. On applique chaque mouvement possible, obtenant de nouvelles valeurs de twist, distance = 1. On continue de proche en proche. Chaque valeur de twist reçoit sa distance minimale.

Le BFS explore tout l'espace des twists (2187 valeurs). C'est rapide. On stocke le résultat dans un tableau de 2187 entrées, chacune contenant un petit entier (la distance, au plus 15 ou 20). Ça tient dans quelques kilo-octets.

6.3 — Combiner plusieurs tables

Une table "twist seul" donne une estimation de la distance uniquement basée sur l'orientation des coins. Mais un état du cube a aussi une valeur de **flip** et de **slice**. On construit donc plusieurs tables :

- Table (twist)
- Table (flip)
- Table (slice)
- Voire des tables combinées : (twist × slice), (flip × slice)...

L'heuristique **h(état)** est le **maximum** des valeurs lues dans ces tables. Pourquoi le maximum ? Parce que chaque table donne un minimum réel (une borne inférieure). Prendre le maximum, c'est prendre la meilleure des bornes.

Analogie visuelle — Trois estimations, on prend la plus optimiste-mais-vraie

Trois amis te disent la distance qu'il te reste pour arriver : "au moins 3 km", "au moins 5 km", "au moins 4 km". Aucun ne ment. Quelle est la meilleure estimation minimale de la distance restante ? 5 km, la plus grande des trois. Les autres sous-estiment.

Le maximum des tables de pruning fonctionne pareil : chaque table dit "il en reste au moins autant", on prend la plus grande des estimations minimales.

6.4 — Les tables de la phase 2

Même principe pour la phase 2. On précalcule les distances aux coordonnées but (permutations résolues). Souvent on utilise des tables combinées comme (corner-perm × slice-perm).

Ces tables sont un peu plus grosses car les coordonnées ont des plages plus larges (40 320 pour les permutations de coins). Mais elles restent gérables.

6.5 — Le compromis mémoire / vitesse

Plus les tables sont grosses (couvrent plus de coordonnées combinées), plus l'heuristique est précise, plus IDA* est rapide. Mais elles prennent plus de mémoire. Un solveur Kociemba classique utilise entre **quelques centaines de Ko** et **quelques Mo** de tables de pruning.

***Idée clé :** Les tables de pruning sont un investissement : on précalcule une fois (quelques secondes ou minutes) pour rendre chaque résolution ultra-rapide (millisecondes). C'est le pattern "cache" appliqué à la recherche de chemin.*

Chapitre 7

Assemblage complet de la machine

Rassemblons tout ce qu'on a vu et regardons ce qui se passe quand tu appuies sur le bouton "résoudre" de ton app.

7.1 — Au démarrage (une seule fois)

Étape A. Construire les tables de mouvements pour les coordonnées de phase 1 (twist, flip, slice) : pour chaque valeur possible $\times$ chaque mouvement, précalculer la nouvelle valeur.

Étape B. Construire les tables de pruning pour la phase 1 (BFS depuis l'état-but).

Étape C. Idem pour la phase 2 : tables de mouvements et tables de pruning.

Ces tables peuvent être stockées dans des fichiers pour ne pas les recalculer à chaque lancement. Un solveur mature charge ces fichiers en mémoire au démarrage.

7.2 — À chaque résolution

Étape 1 (ton Converter). Convertir le cube visuel en état Kociemba (cp, co, ep, eo).

Étape 2. Calculer les 3 coordonnées de phase 1 (twist, flip, slice) depuis cet état.

Étape 3. Lancer IDA* pour la phase 1, en utilisant les tables de mouvements et de pruning. On obtient une séquence de mouvements S1 qui amène le cube dans G1.

Étape 4. Appliquer S1 sur l'état (virtuellement), recalculer les coordonnées de phase 2 (corner-perm, edge-perm-UD, slice-perm).

Étape 5. Lancer IDA* pour la phase 2. On obtient S2.

Étape 6. La solution finale est **S1 + S2**. Tu la renvoies.

Analogie visuelle — La ligne de production

Ton solveur, c'est une chaîne de montage. Le cube visuel entre à un bout, la séquence de mouvements sort à l'autre. Entre les deux : plusieurs postes de travail qui transforment l'information (visuel $\rightarrow$ Kociemba $\rightarrow$ coordonnées $\rightarrow$ recherche $\rightarrow$ séquence). Chaque poste a un rôle clair et s'appuie sur les précédents.

7.3 — L'optimisation classique : plusieurs solutions de phase 1

L'amélioration standard consiste à générer **plusieurs solutions** pour la phase 1 (pas seulement la première trouvée), et pour chacune, calculer une solution de phase 2. On garde au final la solution combinée la plus courte.

Pourquoi ? Parce qu'une phase 1 courte peut mener à une phase 2 longue, et inversement. En essayant plusieurs combinaisons, on trouve un meilleur optimum global.

7.4 — Ce qu'il faut retenir de l'architecture

- Les **coordonnées** sont l'ADN de l'état.
- Les **tables de mouvements** permettent d'appliquer un mouvement en $O(1)$.
- Les **tables de pruning** permettent d'estimer la distance à l'état-but en $O(1)$.
- **IDA*** orchestre la recherche en utilisant ces deux types de tables.
- Deux phases enchaînées, chacune avec ses propres coordonnées et tables.

***Idée clé :** Kociemba est une **machinerie**. Chaque pièce est simple prise isolément. C'est leur assemblage qui produit un solveur ultra-rapide.*

Chapitre 8

Roadmap pour coder

Maintenant que tu as la vision d'ensemble, voici l'ordre dans lequel je te recommande d'attaquer le codage. Chaque étape est **testable indépendamment** — ne passe à la suivante que quand celle en cours est validée.

Étape 1 — La classe CubieCube

Ta première brique : une classe qui représente un état Kociemba (cp, co, ep, eo) et sait appliquer les 6 mouvements de base (U, D, F, B, L, R). Un mouvement se code comme une **permutation** des indices + un ajustement des orientations.

Test : applique U puis U' — tu dois retrouver l'état initial. Applique U 4 fois — pareil.

Test croisé : applique une séquence sur ton Cube visuel **et** sur ta CubieCube (via ton Converter), vérifie que les deux états correspondent après.

Étape 2 — Les coordonnées de phase 1

Ajoute des méthodes qui calculent twist, flip, slice depuis une CubieCube. Et les inverses : reconstruire une CubieCube depuis un triplet.

Test : calcule → applique un mouvement → recalcule. Compare avec appliquer le mouvement directement sur le triplet (via les tables de mouvements qui viennent après).

Étape 3 — Les tables de mouvements de phase 1

Trois tables : `twist_move[2187][18]`, `flip_move[2048][18]`, `slice_move[495][18]`.
Construction : pour chaque valeur de coordonnée, décode → applique le mouvement sur CubieCube → recode.

Test : vérifie que appliquer le mouvement M puis son inverse M' te ramène à la même valeur.

Étape 4 — Les tables de pruning de phase 1

Un ou deux BFS depuis l'état-but pour construire les tables de distance minimale. Commence par des tables simples (une par coordonnée), puis passe éventuellement à des tables combinées si tu veux plus de performance.

Test : la distance de l'état-but à lui-même doit être 0. Toutes les autres doivent être > 0. La distance maximale doit être raisonnable (~12-15).

Étape 5 — IDA* pour la phase 1

L'algorithme lui-même. Un DFS récursif avec profondeur limitée, qui incrémente la limite quand aucune solution n'est trouvée.

Test : lance sur un cube résolu — solution vide. Sur un cube après un seul mouvement F — solution F' (ou F^3) de longueur 1 à 3.

Étape 6 — Répète pour la phase 2

Nouvelles coordonnées (permutations), nouvelles tables de mouvements (uniquement les 10 mouvements de G1 cette fois), nouvelles tables de pruning, IDA* adapté.

Test : à partir d'un état déjà dans G1, la phase 2 seule doit résoudre le cube.

Étape 7 — L'orchestration

Une fonction solve(cubeVisuel) qui : convertit → phase 1 → applique la solution → phase 2 → concatène les solutions.

Test final : lance sur des scrambles connus, vérifie que la solution appliquée sur le cube initial le résout bien.

Combien de temps ça prend

Compte réellement **plusieurs semaines** si tu veux tout coder proprement en comprenant chaque étape. C'est normal. Ne te compare pas à des repos GitHub existants — ils compressent des mois de travail en quelques centaines de lignes.

***Idée clé :** L'ordre importe. Chaque brique s'appuie sur la précédente. Ne saute jamais une étape. Si tu bloques sur l'étape 5, c'est probablement que l'étape 3 ou 4 a un bug caché.*

Épilogue

Tu as maintenant, j'espère, une image mentale claire de ce qu'est Kociemba. Ce n'est pas de la magie noire. C'est une combinaison intelligente d'idées simples :

- Le cube est un **espace** qu'on explore.
- On passe par un **état intermédiaire** (G1) pour couper le problème en deux.
- On **compresse** l'état en quelques entiers (coordonnées).
- On **précalcule** tout ce qu'on peut (tables de mouvements et de pruning).
- On explore avec un **DFS malin** (IDA*) guidé par des heuristiques précalculées.

Chacune de ces idées, prise isolément, est apprivoisable. Assemblées, elles forment un des algorithmes les plus élégants de la résolution combinatoire.

Reprends ce cours quand tu commences à coder. Chaque fois que tu es perdu dans une notion abstraite, reviens à l'analogie visuelle correspondante. Le hall d'entrée, le compteur kilométrique, l'ADN, la carte routière, le GPS avec un oiseau qui parle : ces images sont tes ancres.

Et souviens-toi : si Kociemba lui-même a mis des années à mettre au point cet algorithme, il est normal que toi tu mettes des semaines à le comprendre et à le coder. Ce n'est pas un sprint. C'est une expédition.

Bon voyage.